META GLOBAL AI DEVELOPER HACKATHON · TRACK: BUILD FOR YOUR WORLD

Wardrobe Stylist *Design Spec*

A mobile-first web app that turns a one-minute closet video into a digital wardrobe, then plans what to wear each day — and on trips — from your calendar, the weather, and clothes you actually own.

DATE
26 Oct 2026

TEAM
Arjun Rangarajan (solo, with Claude Code)

BUILD WINDOW
27 Oct – 4 Nov 2026

CONTENTS

1	Intent & success criteria
2	Architecture
3	Data model
4	Closet scan
5	Stylist agent
6	Screens
7	Error handling & testing
8	Demo script & schedule
9	Stretch goals
10	Pitch: what Meta could do

Companion document: **UI Style Guide** (visual direction “The Lookbook”).

1 Intent

The problem, firsthand: “I own plenty of clothes and still wear the same few things.” Every morning is a decision made without knowing what the day holds. Packing for a trip is the same problem multiplied by every day away.

The product: film your closet once and the app builds a digital wardrobe. Each morning it reads your calendar and the weather, asks about anything that isn’t on the calendar, and proposes an outfit from clothes you own — explaining why, and optionally rendering it on you. For trips it plans each day and builds a packing list that respects how often you’re willing to rewear each kind of garment.

Success criteria

- Judges can open a deployed link and use a working product.
- The 3-minute demo shows: closet scan → daily outfit with visible self-correction → “anything else today?” → trip packing → closet gap card.
- At least four Meta models, each doing something distinct: SAM 3.1, Muse Spark, Muse Image, Muse Voice Transcribe (plus Model API search grounding).
- Everything shown was built 27 Oct – 4 Nov, provable from the public repo’s commit history.

How it maps to the judging rubric

Criterion	Weight	Our answer
Impact	30%	A daily, personal problem; less decision fatigue, fewer duplicate purchases
Use of Meta AI tooling	25%	A pipeline across four Meta models, each used for what it’s uniquely good at
Technical depth	20%	A generate → verify → revise agent loop with code-checked rules
Demo	25%	Visual at every step: closet appears, outfit renders, agent corrects itself on screen

Stretch goals (section 9): a “which look?” poll and style from screenshots.

Out of scope for the MVP (designed for, built only if time allows): laundry-tracking UI, the no-repeat-with-same-people rule, a full shopping feed, and plugging into the consumer Meta AI agent (to be asked in Discord office hours as a stretch).

2 Architecture

CLIENT	SERVER (SAME NEXT.JS PROJECT)	EXTERNAL
Next.js (App Router, TypeScript)	API routes · outfit loop · rule engine	Meta Model API
Mobile-first web app	Model adapters (real / mock)	Open-Meteo weather
Camera + mic in the browser	Supabase Postgres + Storage	Vercel hosting
	Google sign-in + Calendar (read-only)	

Model adapters

Every model call goes through a thin interface with a real and a mock implementation, chosen by an environment variable (`MODELS=mock|real`). Mocks return saved fixtures, so the whole app can be built

before API credits arrive.

Adapter	Meta model	Job
`segmenter`	SAM 3.1	Video / photo → a mask and track ID per garment
`tagger`	Muse Spark (vision)	Cutout → structured garment attributes
`stylist`	Muse Spark	Context + wardrobe → candidate outfits (JSON) and revisions
`renderer`	Muse Image	Outfit → try-on or flat-lay image
`transcriber`	Muse Voice Transcribe	Audio → text

If SAM 3.1's API video support is weak: sample frames, segment each, dedupe. Further fallback: SAM 3 open weights.

3 Data model

```
Profile   user_id, home_city, home_lat/lon, person_photo?,
          render_mode: tryon | flatlay | none      (default flatlay)
          wear_limits: { category → max wears before wash | unlimited }

Garment   id, user_id, source_media_id, photo_url, cutout_url,
          category, subcategory, colors[], pattern, formality(1–5), warmth(1–5), tags[],
          status: clean | worn | laundry          (default clean)
          wear_limit_override?, wears_since_wash, wear_count, last_worn_at, confirmed

Outfit    id, user_id, garment_ids[], rationale (per-garment "why"), rule_report,
          revision_log, render_mode, render_url?, source: agent | user, context_date

WearLog   id, user_id, date, outfit_id, event_id?, people[]

Event     id, user_id, date, start, end, title, location?, attendees[], source: calendar | user

Trip      id, user_id, destination, start_date, end_date, day_plans[], packing_list

ScanJob   id, user_id, media_url, status, progress, found_count, error?
```

Default rewear limits (editable, with per-item overrides)	
1 wear	Underwear, socks, t-shirts, sweatpants / athletic
2 wears	Dress shirts
3 wears	Sweaters, trousers
5 wears	Jeans
Unlimited	Outerwear, shoes, accessories

> **Built to extend.** `status` and `wears_since_wash` exist from day one, so laundry tracking is just a UI. `WearLog.people` and `Event.attendees` exist from day one, so the no-repeat rule is just one more rule file. No migrations needed.

4 Closet scan

Capture	`SWEEP` pan across the closet · `HOLD-UP` show items one at a time · `PHOTO` single item. A photo is a one-frame video; all three share one pipeline.
Upload	To Storage; a `ScanJob` reports progress ("Found 14 · tagging 6/14").
Segment — SAM 3.1	Text prompt "clothing item" on the video → each garment tracked across frames under one ID.
Best frame	Per garment, pick the largest and sharpest mask → transparent PNG cutout.
Tag — Muse Spark	Category, subcategory, colors, pattern, formality, warmth, tags — schema-validated, retried once, else left for manual tagging.
Dedupe	Similar candidates compared by Muse Spark ("same item?").
Review	You confirm, fix tags or delete. Only confirmed garments are used by the stylist.

> **Day-1 risk check (when API credits arrive):** run one real closet video through SAM 3.1 and confirm text prompts on video, the mask format and size limits.

5 Stylist agent

The outfit loop — run by our code in a fixed order

Gather context	Events (calendar + ones you added) · weather at the event or home location · eligible garments
Propose	Muse Spark returns 3 candidate outfits as JSON: garment IDs + a "why" for each piece
Check → revise	Every rule checks every candidate. If all fail, the violations go back to Muse Spark to revise (max 2 rounds).
Pick	Rank survivors by rule soft-scores + the model's ranking
Render	Muse Image in your render mode; on failure or timeout, fall back to the piece grid
Store & show	Outfit with its rule report and revision log — shown in the UI as "fitting notes"

Rules

Each rule is one file implementing `{ id, kind: hard | soft, check(outfit, ctx) → violations[], score?(outfit, ctx) }`.
Adding a rule means adding a file to the registry.

Rule	Kind	When	What it checks
completeness	hard	MVP	Top + bottom (or one-piece) + shoes
formality	hard	MVP	Matches the day's most formal event
weather	hard	MVP	Warmth vs. temperature; nothing rain-unsafe at ≥50% rain
availability	hard	MVP	Every piece is clean
wearLimit	hard	Trips	Uses per garment ≤ its rewear limit
capsuleReuse	soft	Trips	Fewer distinct items scores higher
noRepeatWithSamePeople	hard	Later	Not worn recently in front of the same attendees
laundryCycle	hard	Later	Moves items to laundry when they hit their limit

Daily flow

- The day's most formal event sets the dress code. A big gap between events (gym → client meeting) produces a change-of-clothes suggestion.
- After the first outfit appears, the card asks: *"Anything else today that's not on your calendar?"* Quick chips (gym, dinner out, date, outdoors, nothing) or voice / text. Answers become `Event(source: user)` and the loop runs again.
- "I wore this" writes a `WearLog` entry and updates the wear counts.

Chat agent

Muse Spark with tools: `get_events`, `get_weather`, `search_wardrobe`, `plan_outfit`, `plan_trip`, `log_wear`, `add_event`. Voice comes in through the transcriber. Free-form requests become tool calls; outfits always come from the outfit loop, never straight from the chat model.

Trip packing

Input: destination and dates, or detected from calendar events ("Flight to ORD"). For each day, the destination's weather plus any trip events go through the outfit loop with `wearLimit` and `capsuleReuse`. Output: day-by-day outfits, a packing checklist (each garment and how many days it's used), and a warning when your wardrobe can't meet the limits ("you own 3 clean t-shirts for 5 days").

Closet gap card

Muse Spark proposes 3 items you might be missing. Our code counts, for each one, how many new rule-passing outfits it would make possible — the model can't invent that number. The best one is matched to a real product through Model API search grounding. Shown as one card, not a feed.

6 Screens

Screen	Contents
Onboarding	Google sign-in → home city → optional photo of you → render mode → rewear limits
Today	Weather and events line, outfit (render, why, fitting notes), “anything else today?”, I wore this, another look
Closet	Grid, filters, status words, + Scan
Scan	Capture → progress → review
Mic (global)	Chat agent by voice
Trip	Destination + dates → day-by-day outfits + packing checklist
Settings	Render mode, rewear limits, home city

Visual design follows the companion UI Style Guide.

7 Error handling & testing

Error handling

- Every model call has a timeout and a typed error; the UI names the failure and offers a retry.
- A failed render falls back to the piece grid and never blocks the outfit.
- Stylist output is schema-validated; unknown garment IDs count as violations.
- Too few garments for a valid outfit → say which category is missing (feeds the gap card).
- No calendar → manual events only. No weather → ask “warm, mild or cold?”.

Testing

- Unit tests for every rule with fixture outfits.
- Outfit-loop tests with a mock stylist — including one that breaks rules, to exercise revision.
- Trip planner: satisfiable and unsatisfiable rewear limits.
- Adapter contract tests against fixtures; a manual smoke script for the real APIs.
- 3 Nov: run every demo step against the real APIs.

8 Demo script & schedule

Time	Moment
0:00	Hook: “I own 60 pieces of clothing and wear 6 of them.”
0:20	Closet scan: pan the phone, cutouts appear, review
0:50	Today: client meeting + rain → render, and the agent catches its own mistake
1:30	“Anything else today?” → by voice: “drinks after work” → outfit updates
2:00	Trip: “Chicago Thu–Sun” → packing list that respects rewear limits
2:30	Closet gap card with a computed outfit count
2:45	The Meta models used, and that everything was built during the hackathon

Day	Goal
Tue 27 Oct	App skeleton, Supabase, Google sign-in, data model, adapters + mocks
Wed 28 Oct	Closet scan UI + pipeline (mocked) + review screen
Thu 29 Oct	Rule engine + outfit loop + Today screen
Fri 30 Oct	Real calendar + weather, “anything else today?”
Sat 31 Oct	Render modes (try-on / flat lay / none)
Sun 1 Nov	Chat agent + voice
Mon 2 Nov	Trip packing + rewear limits
Tue 3 Nov	Gap card, polish, deploy, full real-API run
Wed 4 Nov	Demo video, README, submit by the afternoon (deadline 9 PM)

The SAM check runs first, whatever day credits arrive. The repo must be public; the README states that all code was written during the hackathon.

9 Stretch goals

Only after the core demo works end to end. Built in this order, each behind a feature flag so an unfinished one never reaches the demo.

9a · "Which look?" poll — about half a day

Create	From Today or a trip day, pick 2–3 looks → a `Poll` with a public link.
Share	On phones, the browser's share sheet sends the poll image + link to Instagram Stories, Messages and so on.
Vote	Friends open the link with no sign-in, see the renders, tap one. One vote per browser (cookie + hashed IP — good enough among friends).
Use the result	The winner gets a fitting note ("Friends picked this, 5–2") and a soft score for that event's outfit.

`Poll` id, user_id, slug, outfit_ids[], event_id?, closes_at, votes { outfit_id → count }

9b · Style from screenshots — about a day

- Upload screenshots of outfits you like — Instagram saves, Pinterest, anything.
- Muse Spark (vision) reads them into a style profile, e.g. "muted earth tones, relaxed tailoring, minimal logos".
- The profile is shown and editable in Settings — never hidden.
- A soft `styleMatch` rule uses it to break ties between outfits that pass every hard rule, and to rank closet-gap suggestions. It never overrides a hard rule.

`StyleProfile` summary, traits[], palette[], avoid[], source_images[]

> **Why not real Instagram likes?** Instagram retired its personal-account API (Basic Display) in December 2024. The Graph API that remains covers only Business and Creator accounts' own content — not what someone liked or saved. Screenshots get the same effect with no API, and the real thing becomes the pitch below.

10 Pitch: what Meta could do with this

A closing slide, framed clearly as future direction — not something built during the hackathon.

Idea	What it would look like
Taste from likes and saves	With consent, Instagram's signals replace screenshot uploads, so the stylist knows your taste from day one.
Polls inside Stories	A native "which look?" sticker instead of a link.
Shop the gap	Closet-gap suggestions go straight to Instagram Shopping, ranked by how many new outfits each item makes possible — useful recommendations, not spam ads.
Ray-Ban Meta glasses	"Hey Meta, what am I wearing?" logs the outfit automatically; a glance at the closet updates the wardrobe.
The Meta AI assistant	The wardrobe becomes a tool the assistant can call: "what should I pack for Lisbon?"